

Exercise 2: E-commerce Platform Search

Function

Big O Notation:

Big O Notation is a mathematical tool used in computer science to describe the performance or complexity of an algorithm. It details how the execution time or space requirements grow as the input data size (n) increases.

Best-case scenario: The algorithm performs the minimum number of operations. For example, finding the target element on the very first attempt.

Average-case scenario: The expected performance when the target element is located randomly within the dataset. For example, the target element is somewhere in between of the array or dataset.

Worst-case scenario: The absolute maximum number of operations required. For example, the target element is at the very end of the dataset or does not exist at all.

Product.java:

```
public class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    public void display() {
        System.out.println(productId + " " + productName + " " + category);
    }
}
```

LinearSearch.java

```

public class LinearSearch {
    public Product search(Product[] products, int key) {
        for (Product p : products) {
            if (p.productId == key) {
                return p;
            }
        }
        return null;
    }
}

```

BinarySearch.java:

```

public class BinarySearch {
    public Product search(Product[] products, int key) {
        int left = 0, right = products.length - 1;

        while (left <= right) {
            int mid = (left + right) / 2;

            if (products[mid].productId == key)
                return products[mid];

            if (products[mid].productId < key)
                left = mid + 1;
            else
                right = mid - 1;
        }

        return null;
    }
}

```

Main.java

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of products: ");
        int n = sc.nextInt();
        sc.nextLine();

        Product[] products = new Product[n];

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Product " + (i + 1));

            System.out.print("Product ID: ");
            int id = sc.nextInt();
            sc.nextLine();

```

```

        System.out.print("Product Name: ");
        String name = sc.nextLine();

        System.out.print("Category: ");
        String category = sc.nextLine();

        products[i] = new Product(id, name, category);
    }

    Arrays.sort(products, Comparator.comparingInt(p -> p.productId));

    System.out.print("\nEnter Product ID to search: ");
    int key = sc.nextInt();

    LinearSearch linear = new LinearSearch();
    Product linearResult = linear.search(products, key);

    System.out.println("\nLinear Search Result:");
    if (linearResult != null) {
        linearResult.display();
    } else {
        System.out.println("Product not found");
    }

    BinarySearch binary = new BinarySearch();
    Product binaryResult = binary.search(products, key);

    System.out.println("\nBinary Search Result:");
    if (binaryResult != null) {
        binaryResult.display();
    } else {
        System.out.println("Product not found");
    }

    sc.close();
}
}

```

```

public Document createDocument() {
    return new ExcelDocument();
}
}

```

Output:

```
cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Data Structures and Algorithm/Exercise2/" && javac Main.java && java Main
adityahargunani@Adityas-MacBook: Cognizant % cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Data Structures and Algorithm/Exercise2/" && javac Main.java && java Main
Enter number of products: 3

Enter details for Product 1
Product ID: 101
Product Name: Laptop
Category: Electronics

Enter details for Product 2
Product ID: 102
Product Name: Shoes
Category: Fashion

Enter details for Product 3
Product ID: 103
Product Name: Watch
Category: Accessories

Enter Product ID to search: 102

Linear Search Result:
Product ID: 102
Product Name: Shoes
Category: Fashion

Binary Search Result:
Product ID: 102
Product Name: Shoes
Category: Fashion
adityahargunani@Adityas-MacBook: Exercise2 %
```

Time Complexity Comparison:

Algorithm	Best-Case	Average-Case	Worst-Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Which is More Suitable?

Binary Search is more suitable for an e-commerce platform because:

- Product catalogs can contain thousands or millions of items.
- Binary search provides $O(\log n)$ performance, making searches significantly faster.